// AI PLATFORM FACTORY — PRODUCTION AGENT RUNBOOK v2.0 | MULTI-SERVICE | FULL-STACK | JUNE 2026

AI Platform Factory

Production-Grade Requirements & Agent Runbook

Full-stack platform | Frontend + Backend + Data pipeline + Agent layer | GitHub Copilot + Claude Code

HOOKS SKILLS AGENTS.md CLAUDE.md Docker DevContainer Data Governance MCP

WHAT CHANGED FROM V1 (POC)

v1 was PoC-scoped (single microservice). v2 covers the full platform: frontend SPA, backend APIs, data pipeline, event bus, agent orchestration layer, Docker production stack, data schema + governance, DevContainer (optional), and production hooks + skills. Every section is machine-readable by GitHub Copilot Agent and Claude Code.

00 TABLE OF CONTENTS

use Ctrl+F / section numbers to navigate

01 HOW TO USE THIS DOCUMENT

GitHub Copilot Agent + Claude Code

Four integration methods — choose by your workflow:

Method A — Attach to Copilot Chat

1. Open GitHub Copilot Chat in VS Code
2. Click paperclip → Upload this PDF
3. Paste the Section 13 bootstrap prompt
4. Copilot reads the PDF as full context

Method B — Copilot Space (persistent)

1. github.com/copilot/spaces → New Space
2. + Add content → Upload this PDF
3. Add your repos to the Space
4. Every chat session inherits this context

Method C — Claude Code CLAUDE.md

1. Save content as CLAUDE.md at repo root
2. Claude Code reads it on every session
3. Symlink: AGENTS.md → CLAUDE.md
4. Both agents share one file

Method D — Commit to requirements/

1. Save as requirements/00-platform-spec.md
2. Add .github/copilot-instructions.md
3. Add AGENTS.md at repo root
4. Both agents auto-discover at session start

COPILOT-INSTRUCTIONS.MD + AGENTS.MD — THE CROSS-AGENT STANDARD

Commit both files. CLAUDE.md is a symlink to AGENTS.md so both agents share one source of truth.

copilot-instructions.md = GitHub Copilot's always-on context. AGENTS.md = all-agent standard.

Keep both under 150 lines. Split into subdirectories at 200 lines to avoid context bloat.

02 PLATFORM ARCHITECTURE OVERVIEW

service map — agent must internalize

A multi-service platform consists of these layers. The agent must understand all of them before touching any code. Use this map to orient in every session.

Layer	Service(s)	Responsibility	Owns
Frontend	web-app (SPA)	User interface, routing, state management, API client	src/frontend/
API Gateway	gateway-svc	Auth, rate-limit, routing fan-out, observability	src/gateway/
Backend APIs	user-svc, order-svc, product-svc…	Business logic, domain ownership, DB per service	src/services/*/
Data pipeline	ingest-svc, transform-svc, export-svc	ETL, event processing, schema validation	src/pipeline/
Event bus	Kafka / NATS / SQS (detected from repo)	Async messaging, event sourcing, CQRS read side	infra/events/
Storage	PostgreSQL, Redis, S3/blob (per svc)	Persistence, cache, object store	infra/db/
Agent layer	orchestrator-agent, specialist-agents	AI task routing, tool use, multi-agent fan-out	src/agents/
Observability	Prometheus, Grafana, Loki, traces	Metrics, logs, distributed traces	infra/observability/
IaC / Infra	Terraform / Pulumi / CDK	Cloud resources, networking, secrets management	infra/

SERVICE-TO-SERVICE COMMUNICATION RULES

Synchronous: REST or gRPC only between gateway and first-tier services. Use gRPC for high-throughput.

Asynchronous: Event bus for cross-domain events. Producer owns the schema. Consumer validates on read.

Never: Direct DB-to-DB access. Never shared schema between services. One service = one bounded context.

03 REPOSITORY STRUCTURE — FULL PLATFORM MONOREPO

Platform monorepo — annotated

```
platform-root/
  AGENTS.md                       # Cross-agent instructions (symlinked to CLAUDE.md)
  CLAUDE.md -> AGENTS.md          # Symlink — Claude Code reads this
  .github/
    copilot-instructions.md        # GitHub Copilot always-on context
    hooks/
      policy.json                  # [SECTION 07] Hook definitions
      scripts/
        security-check.sh/.ps1     # preToolUse enforcement
        audit-log.sh/.ps1          # postToolUse logging
        session-start.sh/.ps1      # sessionStart banner
        quality-gate.sh/.ps1       # agentStop quality enforcement
    skills/                        # [SECTION 06] Project skills (Copilot + Claude Code)
      architecture/SKILL.md
      frontend/SKILL.md + assets/ + references/
      backend/SKILL.md + references/
      api-design/SKILL.md
      data-schema/SKILL.md + references/schema-registry.yaml
      testing/SKILL.md
      code-review/SKILL.md
      security/SKILL.md
      docker/SKILL.md + assets/
      pipeline/SKILL.md
      observability/SKILL.md
      governance/SKILL.md + references/pii-fields.md
    workflows/                     # [SECTION 11] GitHub Actions
      ci-backend.yml  ci-frontend.yml
      ci-pipeline.yml  security-scan.yml  deploy.yml
  requirements/                   # Agent reads FIRST on every session
    00-platform-overview.md        # What we're building
    01-frontend-requirements.md
    02-backend-requirements.md
    03-data-pipeline-requirements.md
    04-agent-layer-requirements.md
    05-non-functional.md           # SLAs, performance, security
    06-stack.md                    # Stack preferences (or 'agent decides')
    07-constraints.md              # Org rules, compliance, API contracts
  docs/
```

04 TECHNOLOGY STACK DETECTION MATRIX

Agent: never hardcode. Inspect requirements/06-stack.md first, then existing src/ files, then apply the detection heuristics below. Document every decision in an ADR.

Service Layer	Detect from	Common choices	ADR trigger

Frontend SPA	requirements/06-stack.md or src/frontend/	React/Next.js, Vue/Nuxt, Angular, SvelteKit	Framework + SSR strategy
API Gateway	src/gateway/ or infra/ files	Kong, Traefik, AWS ALB, Nginx, custom Node	Auth model, rate limit strategy
Backend service	src/services/*/package.json or pom.xml	Node/TypeScript, Java/Spring, Python/FastAPI, Go/Gin	Language per service
Data pipeline	src/pipeline/ or requirements/03-	Apache Kafka + Flink, AWS Glue, Airflow, custom	Streaming vs batch
Event bus	requirements/03- or infra/events/	Kafka, NATS, AWS SQS/SNS, Azure Service Bus	At-least-once vs exactly-once
Primary DB	requirements/02- or db/migrations/	PostgreSQL, MySQL, MongoDB, DynamoDB	Schema per service vs shared
Cache	requirements/05- (latency SLA)	Redis, Memcached, in-memory (small scale)	Eviction policy
Agent framework	requirements/04- or src/agents/	Claude Agent SDK, LangGraph, CrewAI, custom	Orchestration model
IaC	infra/iac/ or requirements/07-	Terraform, Pulumi, AWS CDK, Bicep	State backend
Container runtime	Docker/Podman presence	Docker + Compose, Podman, containerd	Registry choice
CI/CD	.github/workflows/ presence	GitHub Actions (default), Jenkins, CircleCI	Deploy strategy

05 AI AGENT FILE ECOSYSTEM

AGENTS.md + CLAUDE.md + skills + hooks — how they fit together

File / Location	Read by	Lifetime	Purpose	Rule
AGENTS.md (root)	All agents	Always-on, every session	Cross-agent project context: commands, boundaries, conventions	≤150 lines. Human-written. Never auto-generated
CLAUDE.md (root)	Claude Code	Always-on, every session	Symlink to AGENTS.md. Claude's primary instruction file.	Symlink to AGENTS.md CLAUDE.md
.github/copilot-instructions.md	GitHub Copilot	Always-on, every session	Copilot's always-on repo context. Path-specific instructions possible.	Add path-specific instructions as needed
.github/skills/*/SKILL.md	Copilot + Claude Code	On-demand — description match	Reusable skill: instructions + scripts + resources. Loaded when relevant.	See Section 06 for all 12 skills
.github/hooks/policy.json	Copilot CLI + Coding Agent	Every tool call	Enforcement: preToolUse can deny, postToolUse can log. Deterministic.	See Section 07 for all hook types
requirements/*.md	All agents	Read at session start	Source of truth for features, SLAs, constraints	Agent reads in numeric order 00→07
docs/architecture/overview.md	All agents	Read at session start	Current architecture decisions — agent must update after every structural change	Update after every structural change
docs/adr/*.md	All agents	Read at session start	Decision history — agent never contradicts an ADR without a new ADR	One ADR per decision
.ai-state.md	All agents	Read FIRST every session	Agent's session memory: last step, files changed, pending tasks. Updated at end of every session.	Update at session end. Never delete.
docs/data/schema-registry.yaml	All agents	On-demand via data-schema skill	Canonical data schemas — governed, versioned	See Section 10 for governance rules

THE COMPOSABLE SYSTEM (FROM GITHUB COPILOT DOCS)

Instructions = always on. Skills = auto-loaded on description match. Agents = named persona for a session.

Hooks = deterministic enforcement underneath everything. MCP = live external data via tools.

A skill can bundle scripts an agent invokes. A hook can enforce policy regardless of which skill triggered it.

Hooks are enforcement; instructions are advice. Use hooks when a policy MUST NOT be ignored.

06 PRODUCTION SKILLS LIBRARY

12 skills — install via: gh skills install or copy to .github/skills/

INSTALLATION

All skills follow the open SKILL.md standard. They work identically with GitHub Copilot Agent, Claude Code, Gemini CLI, Cursor, and Windsurf. Install from github/awesome-copilot via:

gh skills install github/awesome-copilot <skill-name> (requires gh CLI v2.90+)

Or copy the skill folder to .github/skills/<skill-name>/SKILL.md manually.

Skills are auto-discovered by description matching. Invoke manually with /<skill-name> in chat.

.github/skills/architecture/SKILL.md

```
---
name: architecture
description: >
  Enforce platform architecture standards. Use when making
  structural decisions, designing new services, reviewing
  service boundaries, or writing ADRs.
---
## Rules
- One service = one bounded context. No shared schemas.
- API-first: define OpenAPI before implementation.
- Write ADR for: stack choice, comm pattern, data store,
  auth model, any decision that affects >1 service.
- Event contracts: producer owns schema, consumer validates.
- Health check required on every service: GET /health.
## ADR trigger checklist
■ New service or removal of existing service
■ New database or change of database per service
■ New external dependency (3rd party API, SDK)
■ Change to inter-service communication pattern
■ Any deviation from requirements/*.md
```

.github/skills/frontend/SKILL.md

```
---
name: frontend
description: >
  Frontend SPA standards. Use when creating or modifying
  components, pages, forms, routing, state management,
  API clients, or build configuration.
---
## Component standards
- Named exports. Arrow function components. TypeScript strict.
- Co-located tests: Component.test.tsx next to Component.tsx.
- Accessibility: WCAG 2.2 AA. Use semantic HTML.
- State: local first, then context, then global store.
- API client: generated from openapi.yaml (openapi-ts or similar).
## references/
  design-tokens.md    # Load only when styling
  component-patterns.md  # Load only when building UI
## assets/
  component-template.tsx  # Scaffold for new components
```

.github/skills/backend/SKILL.md

```
---
name: backend
description: >
  Backend service standards. Use when implementing business
  logic, data access, middleware, or service configuration.
---
## Code standards
- Max function length: 25 lines. Single responsibility.
- Dependency injection via constructor. No singletons.
- Errors: typed returns. Never swallow exceptions silently.
- Logging: structured JSON. Include: trace_id, service,
  level, message, duration_ms. NEVER log PII fields.
- Config: environment variables only. Validate at startup.
  Fail fast if required config is missing.
## Stack detection
  IF src/services/<name>/package.json → TypeScript/Node
  IF src/services/<name>/pom.xml → Java/Spring Boot
  IF src/services/<name>/pyproject.toml → Python/FastAPI
  ELSE → write ADR, choose, implement
```

.github/skills/api-design/SKILL.md

```
---
name: api-design
description: >
  REST and gRPC API design. Use when designing endpoints,
  updating OpenAPI spec, or reviewing API contracts.
---
## REST conventions
- Resources: plural nouns. /v1/users /v1/orders
- Verbs: GET(read) POST(create) PUT(replace) PATCH(modify)
- Responses: 201+Location on create. 204 on delete.
- Errors: RFC 7807 Problem Details format.
- Pagination: cursor-based for large collections.
## OpenAPI sync rule (MANDATORY)
ALWAYS update docs/api/openapi.yaml AFTER implementing
any endpoint. Include: request schema, response schemas,
all error codes (400/401/403/404/409/422/500), examples.
## references/
  error-catalogue.md  # Standard error codes + messages
  auth-patterns.md    # JWT, API key, OAuth2 patterns
```

.github/skills/data-schema/SKILL.md

```
---
name: data-schema
description: >
  Data schema design, migrations, and governance. Use when
  creating or modifying database schemas, event schemas,
  or data contracts. ALWAYS load for any DB operation.
---
## Schema governance rules
NEVER drop columns — mark deprecated, migrate in N+2.
NEVER rename columns directly — add new, migrate, drop old.
ALWAYS add migrations as numbered files in infra/db/migrations/.
ALWAYS update docs/data/schema-registry.yaml.
NEVER put PII fields in: logs, error messages, test fixtures.
PII fields are defined in docs/data/pii-inventory.md.
## references/
  schema-registry.yaml   # Canonical — load when working with data
  pii-fields.md          # PII inventory — load for any user data
  migration-checklist.md # Pre-migration safety checklist
```

.github/skills/testing/SKILL.md

```
---
name: testing
description: >
  Test generation and validation. Use at workflow step 10.
  Covers unit, integration, contract, and E2E tests.
---
## Coverage minimums
  Unit: 80% line coverage on business logic layer.
  Integration: happy path + 2 error paths per endpoint.
  Contract: every API change has a consumer-driven test.
  E2E: critical user journeys only (login, core flow, error).
## Test data rules
- No real PII in test fixtures. Use faker/generated data.
- Database tests use transactions that roll back after each test.
- External services: mock at the network boundary.
## Naming convention
  describe('UserService') > it('creates user with valid data')
  describe('UserService') > it('throws on duplicate email')
```

.github/skills/code-review/SKILL.md

```
---
name: code-review
description: >
  Self-review and quality gate. Load at workflow step 11.
  BLOCKING — agent must not proceed if any item fails.
allowed-tools: Bash(grep *) Bash(cat *)
---
## BLOCKING conditions (stop if any true)
✗ Code does not compile or fails linting
✗ Any test is failing or skipped unexpectedly
✗ Secret or credential found in source code
✗ PII field found in log statement or error message
✗ No test exists for new business logic
✗ OpenAPI spec not updated for new/changed endpoint
## Quality checklist
■ All functions < 25 lines, single responsibility
■ No TODO in production code paths
■ README accurately reflects how to run the service
■ .ai-state.md updated with this session's summary
```

.github/skills/security/SKILL.md

```
---
name: security
description: >
  Security review. Load at workflow step 11. Also load
  when implementing auth, handling user input, or modifying
  any code that touches external data.
---
## OWASP Agentic Top 10 checks (2026)
■ No secrets in source. Use env vars + secrets manager.
■ All user inputs validated before use (zod/pydantic/joi).
■ SQL: parameterised statements only. No string concat.
■ Auth middleware on ALL protected routes.
■ Error messages: no stack traces or internal details to client.
■ Dependencies: exact version pins. No ^ or ~ in prod.
■ Agent tools: least-privilege. Deny by default.
■ PII: encrypt at rest and in transit. Inventory in pii-inventory.md.
■ CORS: explicit allowlist. No wildcard in production.
■ Rate limiting at gateway. 429 with Retry-After header.
```

.github/skills/docker/SKILL.md

```
---
name: docker
description: >
  Docker and Docker Compose standards. Use when creating or
  modifying Dockerfiles, compose.yaml, or container config.
---
## Dockerfile standards
- Multi-stage builds: base → devcontainer → builder → production.
- Production stage: distroless or alpine. No shell in prod.
- Run as non-root user (UID 1001).
- Pin base image digests in production (not just tags).
- COPY --chown=appuser:appuser for file ownership.
- Healthcheck in every production Dockerfile.
## compose.yaml standards
- Use compose.yaml (not docker-compose.yml) per 2026 spec.
- Separate: compose.yaml (dev) compose.prod.yaml (prod overrides).
- Health check dependencies: condition: service_healthy.
- Never hardcode secrets. Use secrets: or env_file: .env
## assets/
  Dockerfile.template   # Multi-stage template per language
```

.github/skills/pipeline/SKILL.md

```
---
name: pipeline
description: >
  Data pipeline standards. Use when building or modifying
  ingest, transform, export services or event handlers.
---
## Pipeline contracts
- Ingest: validate schema on entry. Reject invalid. DLQ for errors.
- Transform: idempotent transformations only.
- Export: respect data contracts in docs/data/data-contracts.yaml.
- Events: producer validates against schema before publishing.
- Consumer: validate on read. Never assume schema stability.
## Lineage
- Log: source, transform steps, destination, timestamp, record count.
- Tag PII fields in lineage metadata.
- Emit data quality metrics to observability stack.
```

.github/skills/observability/SKILL.md

```
---
name: observability
description: >
  Logging, metrics, and tracing standards. Use when adding
  new services, endpoints, or background jobs.
---
## Logging
- JSON structured. Fields: timestamp, level, service,
  trace_id, span_id, message, duration_ms, status_code.
- NEVER log: passwords, tokens, PII fields, credit cards.
## Metrics (Prometheus)
- Every service exposes GET /metrics (or :9090/metrics).
- Mandatory: http_requests_total, http_duration_seconds,
  db_query_duration_seconds, queue_depth (for pipeline).
## Tracing
- OpenTelemetry SDK. Propagate trace context across services.
- Instrument: HTTP handlers, DB queries, event publishing.
```

.github/skills/governance/SKILL.md

```
---
name: governance
description: >
  Data governance and compliance. Load when working with
  user data, PII, data contracts, or schema changes.
  Load for any change touching docs/data/.
---
## PII governance rules
NEVER log PII fields (see docs/data/pii-inventory.md).
NEVER include PII in test fixtures.
NEVER return PII in error responses.
ALWAYS encrypt PII at rest (AES-256 or KMS).
ALWAYS use field-level encryption for SSN, credit cards.
## Data contract rules
- Schema changes: write data-contract change proposal first.
- Breaking changes: 14-day notice period minimum.
- Consumers must validate schemas on read.
## references/
  pii-inventory.md     # Canonical PII field list
  data-contracts.yaml  # Service data contracts
```

INSTALL COMMUNITY SKILLS (GITHUB/AWESOME-COPILOT)

$ gh skills install github/awesome-copilot agent-governance

$ gh skills install github/awesome-copilot codebase-mapper

$ gh skills install github/awesome-copilot acreadiness-assess

Or browse: github.com/github/awesome-copilot | lobehub.com/skills

07 PRODUCTION HOOKS

.github/hooks/ — enforcement + audit + quality gates

Hooks are the enforcement layer. Unlike instructions (which the model may ignore), preToolUse hooks deterministically block or allow tool execution. Every hook script must run in under 5 seconds.

Hook event	When fires	Capability	Production use
sessionStart	Agent session begins	Observational only	Log session, banner, load env context
userPromptSubmitted	User sends a prompt	Observational only	Audit log prompt, detect sensitive intent
preToolUse	BEFORE any tool runs	BLOCKING — can allow/deny/ask	Security gate, deny dangerous commands, enforce policy
postToolUse	AFTER tool completes	Can modify result / inject context	Audit trail, auto-format code, run linter
agentStop	Agent finishes responding	Blocking for quality gate	Run test suite, verify quality gates before session ends
subagentStop	Subagent completes	Observational only	Log subagent results, pass context to parent
errorOccurred	Error during execution	Observational only	Error logging, alerting, pattern tracking
sessionEnd	Session terminates	Observational only	Cleanup, send Slack notification, generate session report

policy.json — full production configuration:

```
// .github/hooks/policy.json
{
  "version": 1,
  "hooks": {
    "sessionStart": [{
      "type": "command",
      "bash": "./.github/hooks/scripts/session-start.sh",
      "powershell": "./.github/hooks/scripts/session-start.ps1",
      "timeoutSec": 10
    }],
    "userPromptSubmitted": [{
      "type": "command",
      "bash": "./.github/hooks/scripts/audit-log.sh",
      "powershell": "./.github/hooks/scripts/audit-log.ps1",
      "timeoutSec": 5
    }],
    "preToolUse": [
      {
        "type": "command",
        "bash": "./.github/hooks/scripts/security-check.sh",
        "powershell": "./.github/hooks/scripts/security-check.ps1",
        "comment": "Security gate — runs first, can deny. Must be < 5s.",
        "timeoutSec": 5
      },
      {
        "type": "command",
        "bash": "./.github/hooks/scripts/audit-log.sh",
        "timeoutSec": 3
      }
    ],
    "postToolUse": [{
      "type": "command",
      "bash": "./.github/hooks/scripts/post-tool.sh",
      "timeoutSec": 10
    }],
    "agentStop": [{
      "type": "command",
      "bash": "./.github/hooks/scripts/quality-gate.sh",
```

security-check.sh — preToolUse enforcement script:

```
#!/bin/bash
# .github/hooks/scripts/security-check.sh
# Reads JSON from stdin. Writes allow/deny JSON to stdout.
# MUST run in < 5 seconds.
INPUT=$(cat)
TOOL=$(echo "$INPUT" | jq -r ".toolName // empty")
TOOL_ARGS=$(echo "$INPUT" | jq -r ".toolArgs // empty")

# --- Redact secrets from args before logging ---
REDACTED=$(echo "$TOOL_ARGS" | \
  sed -E "s/ghp_[A-Za-z0-9]{20,}/[REDACTED]/g" | \
  sed -E "s/Bearer [A-Za-z0-9_.-]+/Bearer [REDACTED]/g" | \
  sed -E "s/--password[= ][^ ]+/--password=[REDACTED]/g")

# --- Log every attempt (JSONL audit trail) ---
mkdir -p logs
echo "{"ts":"$(date -u +%Y-%m-%dT%H:%M:%SZ)","tool":"$TOOL"}" >> logs/audit.jsonl

# --- Enforce policy on bash tool only ---
if [ "$TOOL" != "bash" ]; then exit 0; fi
CMD=$(echo "$TOOL_ARGS" | jq -r ".command // empty" 2>/dev/null)

# --- DENY dangerous patterns ---
DENY_PATTERNS=("rm -rf /" "DROP TABLE" "DELETE FROM.*WHERE 1" \
               "curl.*eval" "wget.*bash" "> /etc/" "chmod 777")
for pattern in "${DENY_PATTERNS[@]}"; do
  if echo "$CMD" | grep -qiE "$pattern"; then
    echo "{"permissionDecision":"deny","
    echo " "permissionDecisionReason":"Blocked pattern: $pattern"}"
    exit 0
  fi
done

# --- ASK for sensitive operations ---
ASK_PATTERNS=("kubectl.*delete" "terraform destroy" "helm uninstall")
for pattern in "${ASK_PATTERNS[@]}"; do
  if echo "$CMD" | grep -qiE "$pattern"; then
    echo "{"permissionDecision":"ask"}"
```

08 DOCKER PRODUCTION STACK

multi-stage Dockerfile + compose.yaml

Use multi-stage builds: one Dockerfile per service, four stages. Production images are distroless or alpine — no shell. DevContainer stage reuses the base image to prevent environment drift.

Multi-stage Dockerfile template (TypeScript/Node example):

```
# infra/docker/Dockerfile.node  — copy per service and adapt

# ■■ Stage 1: Base (shared dev + prod foundation) ■■■■■■■■■■■■■
FROM node:22-alpine AS base
WORKDIR /app
COPY package*.json ./
RUN npm ci --ignore-scripts

# ■■ Stage 2: Development / DevContainer ■■■■■■■■■■■■■■■■■■■■■■■
FROM base AS devcontainer
RUN apk add --no-cache git curl jq bash
COPY . .
CMD ["npm", "run", "dev"]

# ■■ Stage 3: Builder ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
FROM base AS builder
COPY . .
RUN npm run build && npm prune --production

# ■■ Stage 4: Production (distroless) ■■■■■■■■■■■■■■■■■■■■■■■■■
FROM gcr.io/distroless/nodejs22-debian12 AS production
WORKDIR /app
USER 1001:1001
COPY --from=builder --chown=1001:1001 /app/dist ./dist
COPY --from=builder --chown=1001:1001 /app/node_modules ./node_modules
COPY --from=builder --chown=1001:1001 /app/package.json .
EXPOSE 3000
HEALTHCHECK --interval=10s --timeout=5s --start-period=30s --retries=3 \
  CMD ["/nodejs/bin/node", "-e", \
    "require('http').get('http://localhost:3000/health', r => process.exit(r.statusCode===200?0:1))"]
CMD ["dist/index.js"]
```

compose.yaml — development orchestration:

```yaml
# compose.yaml  (preferred filename per 2026 Compose spec)
name: platform

services:
  frontend:
    build: { context: ./src/frontend, target: devcontainer }
    ports: ["3001:3001"]
    volumes: ["./src/frontend:/app:cached", "/app/node_modules"]
    develop:
      watch:
        - { action: sync, path: ./src/frontend/src, target: /app/src }
        - { action: rebuild, path: ./src/frontend/package.json }
    depends_on: { gateway: { condition: service_healthy } }
    networks: [frontend-net]

  gateway:
    build: { context: ./src/gateway, target: devcontainer }
    ports: ["8080:8080"]
    environment:
      - NODE_ENV=development
      - JWT_SECRET=${JWT_SECRET}  # from .env — never hardcode
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 10s; timeout: 5s; retries: 3
    depends_on:
      db: { condition: service_healthy }
      cache: { condition: service_started }
    networks: [frontend-net, backend-net]

  user-svc:
    build: { context: ./src/services/user-svc, target: devcontainer }
    ports: ["3010:3010"]
    environment:
      - DATABASE_URL=postgres://dev:dev@db:5432/users
    depends_on: { db: { condition: service_healthy } }
    networks: [backend-net]

  db:
```

09 DEVCONTAINER (OPTIONAL)

Docker not required on developer machine — this is opt-in

DOCKER IS OPTIONAL FOR LOCAL DEVELOPMENT

DevContainers provide consistency but require Docker Desktop or Podman.
Developers without Docker can use: scripts/local-setup.sh (see Section 15).
CI always uses containers. DevContainer is recommended but never blocking.
One devcontainer.json per service OR one root devcontainer.json using Docker Compose.

```jsonc
// .devcontainer/devcontainer.json  (monorepo root — uses Compose)
{
  "name": "Platform Dev",
  "dockerComposeFile": "docker-compose.devcontainer.yml",
  "service": "devcontainer",
  "workspaceFolder": "/workspace",
  "shutdownAction": "stopCompose",
  "features": {
    "ghcr.io/devcontainers/features/git:1": {},
    "ghcr.io/devcontainers/features/github-cli:1": {},
    "ghcr.io/devcontainers/features/node:1": { "version": "22" },
    "ghcr.io/devcontainers/features/docker-in-docker:2": {}
  },
  "customizations": {
    "vscode": {
      "extensions": [
        "GitHub.copilot", "GitHub.copilot-chat",
        "esbenp.prettier-vscode", "dbaeumer.vscode-eslint",
        "ms-azuretools.vscode-docker", "42Crunch.vscode-openapi",
        "mtxr.sqltools", "humao.rest-client"
      ],
      "settings": {
        "editor.formatOnSave": true,
        "editor.defaultFormatter": "esbenp.prettier-vscode",
        "chat.useAgentSkills": true
      }
    }
  },
  "postCreateCommand": "npm run setup",
  "forwardPorts": [3001, 8080, 3010, 5432, 6379],
  "portsAttributes": {
    "3001": { "label": "Frontend" },
    "8080": { "label": "API Gateway" }
  }
}
```

10 DATA SCHEMA + GOVERNANCE

PII inventory · schema registry · data contracts · lineage

Data governance is non-negotiable in production. Every field must be classified. Every schema change must follow the contract lifecycle. PII fields are defined once in pii-inventory.md — the agent treats this as law.

docs/data/schema-registry.yaml — canonical schema format:

```yaml
# docs/data/schema-registry.yaml
# Maintained by agent at workflow step 05. Human review required for PII changes.

version: "1.0.0"
last_updated: "2026-06-23"
steward: "@data-platform-team"

schemas:
  users:
    description: "Core user identity. Owned by user-svc."
    database: "users_db"
    table: "users"
    sla: "99.9% availability"
    owner: "@user-platform"
    version: "2.3.1"
    fields:
      id:          { type: uuid,      pii: false, nullable: false }
      email:       { type: varchar(255), pii: true, encrypted: true }
      name:        { type: varchar(200), pii: true, encrypted: false }
      created_at:  { type: timestamptz, pii: false, nullable: false }
      updated_at:  { type: timestamptz, pii: false, nullable: false }
      deleted_at:  { type: timestamptz, pii: false, nullable: true }
    pii_fields: [email, name]   # Cross-reference: docs/data/pii-inventory.md
    change_policy:
      breaking_notice_days: 14
      migration_strategy: "expand-contract"
      prohibited: ["DROP COLUMN", "RENAME COLUMN", "change type narrowing"]

  orders:
    description: "Order lifecycle. Owned by order-svc."
    fields:
      id:           { type: uuid, pii: false }
      user_id:      { type: uuid, pii: false, fk: users.id }
      total_amount: { type: numeric(12,2), pii: false }
      status:       { type: varchar(50), pii: false }
                    # enum: pending | confirmed | shipped | delivered | cancelled
    business_rules:
      - "total_amount = sum(line_items) excluding refunds processed > 30 days"
```

docs/data/pii-inventory.md — governance anchor:

```
# PII Inventory — GOVERNANCE DOCUMENT
# Human review required for any change. Do not auto-generate.

Fields classified as PII (never log, never test fixture, encrypt at rest):
  user.email            user.name          user.phone_number
  user.date_of_birth    user.address       user.ip_address
  payment.card_token    payment.account_no

Fields classified as SENSITIVE (ask data steward before access):
  user.salary_band      user.performance_rating

DQ threshold: Do not use data assets with DQ score < 85%.
Escalation: data-steward@company.com | Slack: #data-governance

# Agent rules (enforced by governance skill + security hook):
NEVER log PII fields.
NEVER include PII in test fixtures (use faker-generated data).
NEVER return PII in error responses or stack traces.
ALWAYS encrypt PII at rest (AES-256 or cloud KMS).
ALWAYS use field-level encryption for payment fields.
```

Data contract lifecycle (for schema changes):

```
# docs/data/data-contracts.yaml — agent generates from template
contracts:
  - id: "orders-v2-contract"
    producer: "order-svc"
    consumers: ["analytics-svc", "billing-svc"]
    schema_version: "2.0.0"
    sla: { availability: "99.9%", max_latency_hours: 4 }
    change_lifecycle:
      propose:   "Create data-contract PR with change description"
      review:    "Data steward + consumer team sign-off"
      notice:    "14-day advance notice to all consumers"
      migrate:   "Expand-contract pattern: add new → migrate → deprecate old"
      deprecate: "Mark old fields deprecated for 2 release cycles"
      remove:    "Remove only after all consumers have migrated"
```

11 CI/CD PIPELINE

GitHub Actions — agent generates these on first scaffold

```yaml
# .github/workflows/ci-backend.yml  (agent generates per service)
name: Backend CI
on:
  push: { branches: ["main", "develop", "copilot/**"] }
  pull_request: { branches: ["main", "develop"] }

jobs:
  lint-and-type-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: "22", cache: "npm" }
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck

  test:
    needs: lint-and-type-check
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16-alpine
        env: { POSTGRES_USER: test, POSTGRES_PASSWORD: test }
        options: --health-cmd pg_isready --health-interval 5s
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm test -- --coverage
      - uses: codecov/codecov-action@v4

  build-and-scan:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build production image
        run: docker build --target production -t service:${{ github.sha }} .
      - name: Scan image (Trivy)
```

12 AI AGENT WORKFLOW — 15-STEP IDEMPOTENT LOOP

all agents, every session

#	Phase	Step	Action	Output
01	ORIENT	Read .ai-state.md	Session memory from last run. Read FIRST.	Context
02	ORIENT	Read requirements/	All *.md in numeric order. Extract: features, SLAs, constraints, stack.	Platform understanding
03	ORIENT	Read docs/	architecture/, adr/, api/, data/schema-registry.yaml, design/plan.	Decision history
04	ORIENT	Inspect src/ + tests/	Walk source. Identify what exists. Find compile/test failures.	Gap list

05	PLAN	Diff requirements vs impl	List: unimplemented features. Undocumented code. Failing gates.	Priority task list
06	PLAN	Update architecture doc	Update docs/architecture/overview.md if any service/decision changed.	Updated arch doc
07	PLAN	Write ADRs	For each new decision not yet documented, generate ADR from template.	docs/adr/ADR-NNN-*.md
08	PLAN	Update impl plan	Write/update docs/design/implementation-plan.md with next increment.	Updated plan
09	BUILD	Scaffold if empty	If any service src/ is empty: generate full scaffold per layer.	Scaffold for layer
10	BUILD	Implement one increment	Load relevant skills only. Implement next unchecked task. One increment.	New/modified src/
11	BUILD	Generate tests	Write unit + integration + contract tests. Run them. Fix errors.	tests/ + coverage
12	REVIEW	Self-review	Load code-review + security skills. Verify ALL quality gates. BLOCKING.	Review notes
13	DOC	Update docs	Sync README, openapi.yaml, schema-registry.yaml, .ai-state.md.	All docs synced
14	DOC	Update schema registry	If any DB/event schema changed, update docs/data/schema-registry.yaml.	Schema registry
15	REPORT	Produce 10-section report	Structured output: summary, assumptions, arch, files, plan, code, tests, docs, remaining, risks.	Session report

13 MASTER AGENT PROMPTS

copy-paste to start any session

Bootstrap prompt — Session 1 (works for both Copilot and Claude Code):

```
## AI Platform Factory — Agent Bootstrap
## Platform: [YOUR PLATFORM NAME]
## Session type: [First run | Resume | Feature: <name> | Bugfix | Review]

You are a senior Software Architect and AI Coding Agent.
You are building a multi-service platform: frontend SPA + backend APIs
+ data pipeline + agent orchestration layer.
The repository is your ONLY source of truth. Never ask for context
that already exists in a file. Read it.

### STEP 1 — ORIENT (read in this exact order)
1. .ai-state.md              — your memory from last session
2. requirements/*.md         — all files, numeric order (00→07)
3. docs/architecture/overview.md
4. docs/adr/*.md             — all ADRs (honour past decisions)
5. docs/data/schema-registry.yaml  — canonical schemas
6. docs/design/implementation-plan.md

### STEP 2 — ASSESS (answer internally before acting)
- Which platform layer needs work next (frontend/backend/pipeline/agents)?
- What stack is active for each service (detect from src/ files)?
- What is the next unchecked [ ] item in implementation-plan.md?
- Are there blocking issues: compile errors, failing tests, schema drift?

### STEP 3 — EXECUTE (ONE increment per session)
Load skills relevant to this increment from .github/skills/.
Execute workflow steps 5–14 (Section 12).
BLOCKING: Step 12 (self-review) must pass before proceeding to Step 13.
If any quality gate fails, fix it before producing the report.

### STEP 4 — REPORT (mandatory 10 sections)
1. Requirements Summary  2. Assumptions  3. Architecture Updates
4. Files Created/Modified  5. Implementation Plan (with [x] markers)
6. Generated Code (key excerpts)  7. Tests (status)
8. Documentation Updates  9. Remaining Work  10. Risks

### PLATFORM-WIDE HARD RULES
NEVER hardcode secrets. NEVER log PII fields.
```

Resume prompt — Sessions 2+ (short version):

```
## Resume — AI Platform Factory
## Platform: [YOUR PLATFORM NAME]

Read .ai-state.md first. Then requirements/. Then docs/. Then src/.
Understand current state without me explaining it.
Find the next [ ] item in docs/design/implementation-plan.md.
Implement it. Test it. Self-review it (quality gates must pass).
Update all docs including schema-registry.yaml if schema changed.
Update .ai-state.md. Produce the 10-section report.
```

Layer-specific prompts (use when targeting a specific service):

```
# Frontend increment:
Load the frontend skill. Focus on src/frontend/. Read docs/api/openapi.yaml first
to understand the API contract before building any UI component.

# Backend service increment:
Load backend + api-design + data-schema skills. Focus on src/services/<name>/.
Read schema-registry.yaml before touching any database code.

# Data pipeline increment:
Load pipeline + data-schema + governance skills. Focus on src/pipeline/.
Update data-contracts.yaml if any schema changes are needed.

# Agent orchestration increment:
Load architecture + backend skills. Read requirements/04-agent-layer-requirements.md.
Document multi-agent routing decisions in an ADR before implementing.

# Security review:
Load security + governance skills. Run OWASP Agentic Top 10 checklist.
Check PII compliance in every service that was modified this sprint.
```

14 QUALITY GATES + SECURITY CHECKLIST

all must pass before session is marked complete

Gate	Requirement	Enforced by
Compile	All services compile with no errors	agentStop hook + CI lint job
Tests	All unit + integration tests pass. Coverage >= 80% on business logic.	agentStop hook + CI test job
OpenAPI	docs/api/openapi.yaml matches all implemented endpoints	code-review skill step 12
Schema registry	docs/data/schema-registry.yaml matches DB migrations	data-schema skill step 14
No secrets	No hardcoded secrets in any file (hook pattern scan)	preToolUse security hook
No PII in logs	grep audit: no PII fields in log statements	security skill step 12
ADR coverage	Every structural decision has an ADR	code-review skill step 12
.ai-state.md	Updated with session summary, files changed, next steps	agentStop hook
Security scan	Trivy + CodeQL + Trufflehog pass (no CRITICAL/HIGH)	CI security-scan.yml

Docker build	Production Dockerfile builds successfully (--target production)	CI build-and-scan job

15 LOCAL TESTING SCRIPTS

no Docker required — for developer machines without container runtime

DEVCONTAINER VS SCRIPTS — BOTH ARE VALID

scripts/ provides non-Docker alternatives for all common tasks.
Developers can run tests, seed databases, and check health without Docker.
CI always runs in containers. Local scripts are developer ergonomics.

```
#!/bin/bash
# scripts/local-setup.sh — install deps for all services without Docker
set -e

echo "→ Checking Node.js version (requires 22+)"
node -v | grep -E "^v(2[2-9]|[3-9][0-9])" || { echo "Node 22+ required"; exit 1; }

echo "→ Installing service dependencies"
for dir in src/frontend src/gateway src/services/*/; do
  if [ -f "$dir/package.json" ]; then
    echo "  Installing: $dir"
    (cd "$dir" && npm ci --silent)
  fi
done

echo "→ Checking database (PostgreSQL must be running locally or via cloud)"
if command -v psql &> /dev/null; then
  psql "$DATABASE_URL" -c "SELECT 1" > /dev/null 2>&1 \
    && echo "  DB: connected" || echo "  DB: not reachable (set DATABASE_URL)"
fi

echo "✓ Local setup complete. Run: scripts/run-tests.sh"

# scripts/run-tests.sh — run all tests without Docker
#!/bin/bash
set -e
echo "→ Running unit tests for all services"
for dir in src/*/; do
  if [ -f "$dir/package.json" ]; then
    (cd "$dir" && npm test -- --passWithNoTests) || exit 1
  fi
done
echo "✓ All tests passed"

# scripts/check-health.sh — check service health endpoints
#!/bin/bash
SERVICES=("3001:frontend" "8080:gateway" "3010:user-svc")
for entry in "${SERVICES[@]}"; do
```